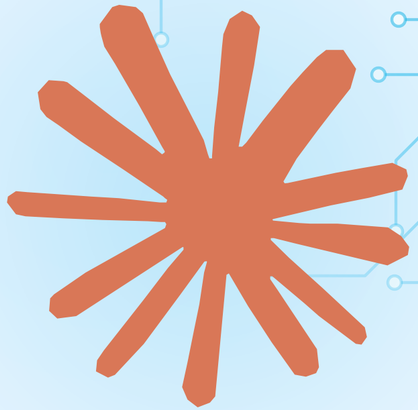
Claude ANTHROP\C

PROGRAMA DE CAPACITACIÓN

Desarrollando con IA

Unidad 2 · Desarrollo agéntico y Spec-Driven Development

Día 1 · Bloque tarde · Instructor: Felipe Cuevas



felipecuevas.dev

RECORRIDO · NIVEL 10 → 20

Dónde estamos

✓ U1	U2 · aquí	U3	U4	U5	U6	U7	U8	U9	U10

ESTA MAÑANA · U1

Mapeaste dónde ayuda la IA

Y dónde decide una persona, en tu docs/mapa-sdlc.md.

ESTA TARDE · U2

Aprendes a guiar al agente

Con una especificación, en tu primera feature del portal Pedidos.

Stack del taller: **ASP.NET Core (.NET 10)** **Rider** **xUnit** **Claude Code**

UNIDAD 2 · HOY

Cuatro ideas para esta tarde

C1

Agente

Piensa, actúa, verifica.

C2

Contexto

Lo que el agente sabe, tú lo decides.

C3

Spec primero

Primero el qué, luego el cómo.

C4

Tú firmas

Si no leíste el diff, no hay commit.

Esta unidad cubre parte del módulo de **agentes y tool use** de Claude Certified Developer – Foundations.

CONCEPTO 1 · 15 MIN

El Agente

01

Piensa, actúa, verifica.

- Ingeniería de contexto
- Ciclo del agente
- Modos de permiso
- Plan mode

De ingeniería de prompts a ingeniería de contexto

Ingeniería de prompts

- Buscas la frase perfecta
- Todo depende de un solo mensaje
- El agente adivina lo que falta

Ingeniería de contexto

- Preparas lo que el agente necesita saber
- Reglas del proyecto, spec y archivos clave
- El agente decide con datos, no con supuestos

Con mal contexto, el agente se equivoca con total seguridad. Tu trabajo es darle el contexto correcto.

El ciclo del agente

1 · Reunir contexto

Lee archivos, busca en el código y pregunta lo que falta.

2 · Actuar

Edita archivos y ejecuta comandos en tu proyecto.

3 · Verificar

Corre los tests, revisa el resultado y corrige.

↻ Repite hasta cumplir el objetivo

SUS HERRAMIENTAS Leer · Editar · Ejecutar

Fuente: Claude Code 101 · «How Claude Code works» (Claude Academy)

C1 · EL AGENTE

Tres modos de permiso

Aprobación manual

Te pide permiso antes de editar o ejecutar.

Ideal para empezar

Auto-aceptar ediciones

Edita sin preguntar; tú revisas el resultado después.

Para tareas acotadas

Plan mode

Solo lectura: investiga y te propone un plan. No toca archivos.

El que más usaremos hoy

`Shift + Tab` cambia de modo hasta ver ⏸ **plan mode on** · o al iniciar: `claude --permission-mode plan`

Fuente: Claude Code Docs · Common workflows y Permission modes

DEMO EN VIVO · 5 MIN · APP FICTICIA TODOMINI

Plan mode en vivo

```
⏸ plan mode on

> Agrega «fecha de vencimiento» a las
tareas. Primero muéstrame tu plan.
```

QUÉ CRITICAR DEL PLAN

- ¿Qué pasa con las tareas sin fecha?
- ¿Formato de fecha y zona horaria?
- ¿Toca archivos que no debía?
- ¿Incluye tests?

1 Pedir el plan
Claude explora y propone, sin editar nada.

2 El curso lo critica
¿Qué supuso? ¿Qué le falta?

3 Lo corregimos
Le respondemos con las decisiones que faltan.

4 Recién entonces, se ejecuta
Salimos de plan mode y aprobamos.

CONCEPTO 2 · 15 MIN

El Contexto

02

Lo que el agente sabe, tú lo decides.

AGENTS.md

/context

/compact

/clear

AGENTS.md: la memoria del proyecto

```
# AGENTS.md — TiendaMini
## Comandos
- Compilar: dotnet build
- Tests: dotnet test
## Convenciones
- Montos en CLP como int, nunca decimal
- Tests: Metodo_Escenario_Resultado
## Reglas SDD
1. Sin spec aprobada, no hay código
2. Empieza en plan mode
3. Si la spec es ambigua, pregunta
```

Qué va

Comandos, convenciones y reglas que el agente no puede adivinar.

Qué no va

Secretos, obviedades y manuales largos. Corto y concreto se cumple mejor.

⚠ **Ojo con /init:** genera un CLAUDE.md. Si existe un CLAUDE.md, Claude Code deja de leer el AGENTS.md (salvo que en /config elijas cargar ambos).

Fuentes: repo u2-sdd-demo (AGENTS.md) · Claude Code Docs, «How Claude remembers your project»

C2 · EL CONTEXTO

Administra la ventana de contexto

/context

Muestra cuánto ocupa cada cosa en la ventana, como una grilla de colores.

Cuándo: antes de una tarea grande

/compact

Resume la conversación para liberar espacio, sin perder el hilo.

Cuándo: sesión larga, misma tarea

/clear

Empieza una conversación nueva con el contexto vacío.

Cuándo: cambias de tarea

Regla práctica: una historia, una sesión. Terminaste la HU-01 y pasas a otra: /clear.

Fuentes: Claude Code 101 · Best practices · Claude Code Docs, «Commands»

DEMO EN VIVO · 4 MIN

El mismo pedido, con y sin AGENTS.md

Pedido: `«Agrega el cálculo de despacho al carrito»`

⚠ Sin AGENTS.md

- Usa decimal para el dinero
- Lee la hora con DateTime.Now
- Nombra los tests a su manera
- Empieza a programar de inmediato

✓ Con AGENTS.md

- Montos en CLP como int
- La hora desde TimeProvider
- Tests Metodo_Escenario_Resultado
- Pide la spec y parte en plan mode

Cierra con **/context**: ¿cuánto de la ventana ocupa el AGENTS.md?

CONCEPTO 3 · 25 MIN

Spec primero

03

Primero el qué, luego el cómo. El núcleo de esta unidad.

- intent → spec → plan → tasks
- Criterios de aceptación
- Caso TiendaMini

La cadena SDD: cuatro artefactos

`intent.md`

Qué queremos y por qué

Escribe: PO

→

`spec.md`

Reglas y criterios verificables

Escriben: PO + dev

→

`plan.md`

Cómo se va a construir

Agente en plan mode · revisa el dev

→

`tasks.md`

Pasos chicos, cada uno verificable

Agente · revisa el dev

☆ **La spec es el artefacto principal: los criterios de verificación viven dentro de ella.**

Con Scrum: cada historia del sprint tiene su spec →

`specs/HU-01-listar-pedidos/spec.md`

Otros enfoques: GitHub Spec Kit (terceros) usa la misma idea con sus plantillas.

Fuentes: AI-native SDLC playbook · Best practices · repo u2-sdd-demo

Anatomía de una spec

1 Contexto y objetivo
Por qué existe esta funcionalidad.

2 Historia de usuario
Como…, quiero…, para…

3 Reglas de negocio
RN-01, RN-02… sin ambigüedad.

4 Criterios de aceptación
CA-xx verificables: Dado / Cuando / Entonces.

5 Ejemplos de entrada y salida
Al menos uno de error.

6 Errores esperados
Código, HTTP y mensaje.

7 Fuera de alcance
Lo que NO se construye.

8 Preguntas abiertas
Lo que falta decidir y quién decide.

9 Historial de cambios
Versión, fecha y quién aprobó.

Fuente: plantillas/spec-plantilla-comentada.md · repo u2-sdd-demo

C3 · CRITERIOS DE ACEPTACIÓN

Un criterio que se puede probar

⚠ Criterio flojo

«El cupón de bienvenida debe funcionar bien.»

· ¿Qué es «bien»?

· ¿Para qué cliente y qué carrito?

· ¿Qué resultado exacto esperamos?

✓ CA-08 · Cupón en primera compra

Dado	un cliente con 0 pedidos pagados y un carrito con subtotal $33.480
Cuando	aplica el cupón BIENVENIDA10
Entonces	descuento 3.348, despacho 3.990 y total 34.122

💡 **Si no sabes cómo escribir el test de un criterio, el criterio todavía no está bien escrito.**

Fuente: ejemplo-carrito/2-spec.md · repo u2-sdd-demo

DEMO EN VIVO · 8 MIN · TIENDAMINI

Prompt suelto frente a spec

«Haz un carrito con descuentos.»		La spec ya lo decidió
¿Qué descuentos existen y cuánto valen?	→	Solo BIENVENIDA10: 10% sobre el subtotal
¿Cómo sabe si es la primera compra?	→	Del historial en el servidor, nunca del request
¿Cupón y despacho gratis se suman?	→	No: se aplica solo el beneficio mayor
¿Cómo se redondea el 10% de $12.345?	→	Se trunca al peso entero: $1.234
¿Qué pasa con la unidad número 11?	→	Se rechaza y el carrito no cambia
¿Cómo se informa un cupón inválido?	→	HTTP 422 con código y mensaje

Fuente: ejemplo-carrito/0-prompt-suelto.md y 2-spec.md · repo u2-sdd-demo

El plan también se revisa

PROPUESTA DEL AGENTE

Calcular el 10% con decimal y Math.Round.

REVISIÓN DEL DEV

Math.Round redondea «al par» y rompe RN-05 (truncar). Usar enteros: subtotal * 10 / 100, y un test con $12.355 → $1.235.

PROPUESTA DEL AGENTE

Que el request traiga esPrimeraCompra: «el front ya lo sabe».

REVISIÓN DEL DEV

Falla de seguridad: cualquiera enviaría true. RN-03 dice que el dato sale del historial en el servidor.

Las dos correcciones salen de leer la spec. Corregir el plan cuesta minutos; corregir el código, horas.

Fuente: ejemplo-carrito/3-plan.md · repo u2-sdd-demo

C3 · TRAZABILIDAD

Cada criterio termina en un test

CA	Qué verifica	Tarea	Test xUnit (método)
CA-03	11 unidades se rechazan	T1	Agregar_OnceUnidades_DevuelveCantidadMaximaExcedida
CA-06	Subtotal 50.000 → despacho gratis	T2	Despacho_Subtotal50000_EsGratis
CA-09	Cupón gana al despacho gratis	T5	Cupon_Subtotal60000_GanaCuponYCobraDespacho
CA-10	10% de 12.345 → 1.234	T5	Cupon_Subtotal12345_DescuentoSeTrunca

RECÓRRELO AL REVÉS: **Test** → **Tarea T5** → **CA-09** → **Regla RN-07** → **Intent**

</> Cada test lleva el ID del criterio en un comentario: `// CA-09`. Si falla, sabes qué regla se rompió.

Fuente: ejemplo-carrito/4-tasks.md y 5-trazabilidad.md · repo u2-sdd-demo

CONCEPTO 4 · 15 MIN

Tú firmas

04

Si no leíste el diff, no hay commit.

Arquitecto y revisor

Checklist del diff

Caza el error

C4 · TÚ FIRMAS

De autor a arquitecto y revisor

El agente escribe el código

Tú diseñas, revisas y decides

Tu firma en el commit dice: «leí esto y respondo por él».

1. ¿Cumple la spec, criterio por criterio?
2. ¿Los tests prueban algo real?
3. ¿Hay casos borde sin cubrir?
4. ¿Hay secretos en el código?
5. ¿Cambió algo que no pedí?

Fuente: Claude Code 101 · flujo explore → plan → code → commit

ACTIVIDAD EN PAREJAS · 6 MIN

Caza el error en el diff

```
src/TiendaMini.Api/Carrito/Carrito.cs
+ private const string PagosApiKey = "tm_live_51Hx9QeV2kLm8Rp";
+ public Resultado Agregar(Producto producto, int cantidad)
+ {
+     if (cantidad < 1) return Error("CANTIDAD_INVALIDA");
+     if (cantidad > 10) return Error("CANTIDAD_MAXIMA_EXCEDIDA");
+     var linea = ObtenerOCrearLinea(producto);
+     linea.Cantidad += cantidad;
+     return Ok();
+ }
tests/TiendaMini.Tests/Carrito/CarritoTests.cs
+ [Fact] // CA-03
+ public void Agregar_OnceUnidades_DevuelveCantidadMaximaExcedida()
+ {
+     var resultado = _carrito.Agregar(_taza, 11);
+     Assert.NotNull(resultado);
+ }
```

3

problemas escondidos en este diff

- Usa la checklist de 5 preguntas
- Compara con la spec de TiendaMini
- Gana la pareja más rápida con los 3

SOLUCIÓN

Los tres hallazgos

```
src/TiendaMini.Api/Carrito/Carrito.cs
+ private const string PagosApiKey = "tm_live_51Hx9QeV2kLm8Rp"; ← 1
+ public Resultado Agregar(Producto producto, int cantidad)
+ {
+     if (cantidad < 1) return Error("CANTIDAD_INVALIDA");
+     if (cantidad > 10) return Error("CANTIDAD_MAXIMA_EXCEDIDA"); ← 2
+     var linea = ObtenerOCrearLinea(producto);
+     linea.Cantidad += cantidad;
+     return Ok();
+ }
tests/TiendaMini.Tests/Carrito/CarritoTests.cs
+ [Fact] // CA-03
+ public void Agregar_OnceUnidades_DevuelveCantidadMaximaExcedida()
+ {
+     var resultado = _carrito.Agregar(_taza, 11);
+     Assert.NotNull(resultado); ← 3
+ }
```

1 · Secreto en el código

Muévelo a un secreto o variable de entorno y rota la clave.

2 · Caso borde sin cubrir

8 + 3 unidades pasa: falta sumar lo que ya hay (CA-04).

3 · Test que siempre pasa

NotNull pasa aunque haya error: verifica el código y el carrito.

REPASO · 4 MIN

Completa la frase

C1 Un agente _____, _____ y _____.

C2 Lo que el agente _____, tú lo _____.

C3 Primero el _____, luego el _____.

C4 Si no leíste el _____, no hay _____.

PARA PROFUNDIZAR

Sigue aprendiendo por tu cuenta

Claude Code

AGENTS.md y su convivencia con CLAUDE.md

Subagentes incorporados: Explore y Plan

Claude Code en la web y control remoto

MCP como concepto (lo veremos en la U9)

Cursos y frameworks

AI-native SDLC: intent.md, requisitos y plan mode

Claude Code in action: sesiones largas

GitHub Spec Kit (terceros): enfoque y diferencias

Libros

Osmani · Beyond Vibe Coding (O'Reilly, 2025)

Martinelli · Spec-Driven Development: From Specs to Code with AI Agents (Apress, 2026)

Enlaces y material en el sitio del curso: ffelipecuevasc.github.io/curso-ia-desarrolladores

15'

Receso

A la vuelta: taller «Mi primera feature guiada por spec». Abre Rider con el repo pedidos-web.

TALLER · 80 MIN · INDIVIDUAL

Mi primera feature guiada por spec

La historia (vaga) del PO

«Como vendedor, quiero listar pedidos filtrando por cliente, estado y rango de fechas.»

¿Qué estados existen? ¿Fechas invertidas? ¿Cuántos resultados? Tú decides y lo escribes en la spec.

Entregable

`AGENTS.md · spec.md · plan.md`

Código, tests xUnit en verde y commit.

1 · Prompt maestro · 10′ **2 · Completar la spec · 15′** **3 · Plan mode · 15′** **4 · Código + tests · 25′** **5 · Revisar el diff · 10′**

6 · Cierre · 5′

Descarga la guía del taller (PDF)

ffelipecuevasc.github.io/curso-ia-desarrolladores → Unidad 2

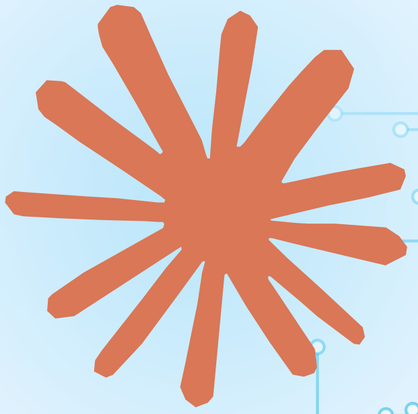
MAÑANA · DÍA 2

Unidad 3 · Generar, refactorizar y documentar

✓ U1 · Mapa SDLC

✓ U2 · Flujo SDD completo

Hoy definiste dónde ayuda la IA y aprendiste a guiarla con una spec.
Mañana construimos en pasos chicos, con tests y Skills.



felipecuevas.dev · ffelipecuevasc.github.io/curso-ia-desarrolladores